Automated Bootstrapping Placement for Encrypted Inference

CS5491 Artificial Intelligence Project Proposal

Member 1: Xing Chunjie | Student ID: 72655910
Member 2: Yang Mianji | Student ID: 72655251
Member 3: Zhang Qi | Student ID: 72655380
Contact email: 72655910@cityu-dg.edu.cn

Problem and motivation

We optimise bootstrapping placement in two supplied neural-network computation graphs: a multilayer perceptron (MLP) block and an attention block. Graph edges carry encrypted data (ciphertexts); their levels track the remaining capacity for multiplication in the course model. Bootstrapping refreshes those levels at substantial cost. One refresh may serve several branches, and ordinary operation costs also depend on levels, so fewer refreshes need not mean lower latency. We minimise the summed estimated execution time of all operations while preserving the original computation and meeting every operation's level requirements.

Method and rationale

We use the Constraint Programming - Satisfiability (CP-SAT) solver in Google's OR-Tools optimisation toolkit. It combines constraints on integer values with logical reasoning about true/false choices to minimise cost while satisfying all specified rules. Our model jointly chooses ciphertext levels and refresh or level-lowering insertions, matching the problem's finite choices. Bootstrap1 refreshes each ciphertext to level 14; Bootstrap2 refreshes a pair to level 13 and is eligible only for edges carrying 8 or 16 ciphertexts. Modswitch lowers levels at zero modelled cost. We enforce operation and refresh requirements and charge each shared insertion once. The instructor supplies graphs, latency functions and the scoring function; we implemented the optimisation model, graph reconstruction and independent structural and level checks. The validated instructor placement guides the search and sets a cost ceiling without excluding better solutions. CP-SAT also supplies a lower bound on cost; when a valid placement reaches this bound, no cheaper placement is possible within the encoded model.

Evaluation approach

We compare against the instructor's fixed placement on each graph at input levels 0-14, with maximum level 14: 30 conditions on two graphs. At each input level, both methods use valid placements and the same complete cost model. We report estimated latency (ms) and its percentage reduction against the matching baseline. Refresh counts n1 and n2 sum the ciphertexts handled by distinct Bootstrap1 nodes and the pairs handled by distinct Bootstrap2 nodes; total count is n1 + n2. Depth is the largest sum of ordinary-operation level consumption along a path; the course-defined budget is input level + 14n1 + 13n2, and utilisation is depth/budget. We separately record time spent finding a placement, sampled process memory, solution status and bounds. Saved graphs and complete diagrams for two input levels per block document the placements.

Progress and remaining work

We calibrated all 30 baselines and completed one optimisation run per condition under a fixed protocol. All reconstructed placements passed independent checks and instructor scoring. Estimated-latency reductions are 7.32-22.11% for MLP and 14.68-22.44% for attention; every solution's cost matched its lower bound. Small examples were also checked by enumerating all allowed placements. Resource records and four complete placement diagrams are prepared. These model estimates do not establish real encrypted-inference or cross-device performance. Remaining work is reproduction in an independent environment and final report preparation.